MODULE 25 · DOMAIN 3 - DEPLOYMENT AND ORCHESTRATION

Deploying foundation models

On-demand, Provisioned Throughput, custom model deployment and Custom Model Import.

MODULE 25 of 43 DOMAIN 3 - Deploy WEIGHT 24%

admin@sthithapragna.com

Deploying a foundation model is a **capacity decision**, not an infrastructure one.

Four ways a foundation model gets served

PATH	WHAT IT IS
On-demand	Pay per token against a shared quota. No commitment, no reservation
Provisioned Throughput	Dedicated capacity in model units, optionally with a term commitment
Custom model deployment	A model you fine-tuned, served on demand where supported
Custom Model Import	Weights you bring, hosted by Bedrock in Custom Model Units

ON THE EXAM

Plus the fifth option from module 20: host an open model yourself on a SageMaker AI endpoint, and manage the instances.

Model access and Region availability

- Model access is requested **per model, per Region**. A model you can call in one Region may be unavailable in another.
- Not every model exists in every Region, and neither does every capability.
- **Data residency from module 08 can pin you to a Region** that lacks the model you wanted.
- Check availability before designing, not after. This is a hard constraint, not a preference.
- IAM policies on the model ID control who may call what, from module 16.

> ON THE EXAM
>
> The unpleasant combination: a residency requirement that pins a Region, and the chosen model not being available there.

On-demand inference

- Pay per input token and per output token. **No commitment, and no idle cost.**
- Capacity comes from a **shared quota**, not reserved for you, so it can be throttled.
- Right whenever traffic is uncertain, which is most of the time.
- Available for base models, and for custom models where supported.
- There is no ceiling on cost either, which module 12 flagged as the other side of that coin.

> ON THE EXAM
>
> On-demand is the sensible default. Reserve capacity only once you can describe your traffic floor with confidence.

Batch inference for foundation models

- Submit a JSONL file of prompts to S3, get a file of responses back.
- **Cheaper per token** than on-demand, and it runs under **separate quotas**.
- So a large batch job does not consume the capacity your live application needs.
- Right for bulk classification, bulk summarisation, and embedding a whole corpus.
- Wrong for anything a user is waiting on. Module 20's question applies here too.

> ON THE EXAM
>
> The separate quota is the underrated part. Moving bulk work to batch relieves the pressure on your real-time allowance.

TPM and RPM quotas

- **Tokens per minute** controls total throughput. **Requests per minute** controls call volume.
- They apply **per model, per Region, per account**.
- They are the **combined sum** across Converse, ConverseStream, InvokeModel and InvokeModelWithResponseStream.
- **Batch inference has separate quotas**, so a batch job does not consume your real-time capacity.
- **Cross-Region InvokeModel tokens per minute** is a separate quota again.

> ON THE EXAM
>
> Two quotas, two different failures. Hitting RPM means too many calls; hitting TPM means too much text, from too few calls or too many.

Spending less of your quota

- **Prompt caching** reuses a repeated prefix, cutting cached tokens sharply. Module 12.
- **Intelligent Prompt Routing** sends easy requests to a cheaper model, from module 12.
- **Shorter prompts and smaller top-k** reduce input tokens directly. Module 10's budget point.
- **maxTokens** caps output, which is usually the dearer half. Module 16.
- Every one of these reduces quota consumption as well as cost. Same lever, two benefits.

ON THE EXAM

Before buying capacity, reduce what you use. Halving your token consumption doubles your effective quota for free.

Throttling, and what to do about it

- Exceeding a quota returns a **throttling exception**. The request is rejected, not queued.
- **Retry with exponential backoff** is the first line of defence, and it handles most bursts.
- **Request a quota increase** through Service Quotas. Increases can be granted across related quotas together.
- **Cross-Region inference profiles** spread load across Regions, raising effective throughput. Module 16.
- AWS gives priority to accounts already consuming their existing allocation. Unused quota weakens the case.

ON THE EXAM

Four responses in order: back off, spread across Regions, request more, then reserve capacity. Reserving is the last step, not the first.

Latency-optimised inference

- Some models offer a **latency-optimised** invocation mode on Bedrock.
- It reduces **time to first token** and overall response time, at a higher per-token price.
- Right when a human is reading the output and waiting visibly for it.
- Combine with **streaming**, from module 16, so the first words appear sooner still.
- Availability is model-specific and Region-specific. Check rather than assume.

ON THE EXAM

Latency and cost trade directly here, as they do everywhere. The question is whether anyone is watching the screen.

Provisioned Throughput

- Dedicated capacity for a specific model, measured in **model units**.
- **A model unit specifies the total input and output tokens it can process within one minute.**
- Predictable throughput, and predictable cost, because you pay for the capacity regardless of use.
- **By default your account has no model units** for committed Provisioned Throughput. Request them through Support first.
- Available for base models and for custom models.

> ON THE EXAM
>
> The unit is throughput, not hardware. You are buying a guaranteed tokens-per-minute rate, not a machine.

Three commitment levels

TERM	WHAT IT MEANS
No commitment	Delete it at any time. Highest hourly price
1 month	Cannot be deleted until the month is over
6 months	Cannot be deleted until six months are over. Lowest hourly price
Billing	Continues until you delete it. Deleting is the only way to stop it
Custom models	Support all three levels. Base model support for no-commitment varies

ON THE EXAM

A committed term cannot be cancelled early. That makes it a genuine financial decision, not a configuration setting.

How many model units do you need?

- Run on demand and record your **actual tokens per minute** at peak, not your estimate.
- Include **input and output together**, because that is what a model unit measures.
- Retrieved chunks and conversation history count as input tokens. Modules 10 and 16.
- Size to the **peak you must serve**, then decide whether to burst beyond it on demand.
- AWS does not publish a universal tokens-per-MU figure. Ask your account team.

ON THE EXAM

The commonest sizing error is counting prompts rather than tokens. A RAG prompt with eight retrieved chunks is not one small request.

Monitoring Bedrock usage

SIGNAL	WHAT IT TELLS YOU
Invocation count	Request volume, per model and per Region
Input and output token counts	Where your spend actually goes. Output is usually dearer
Throttling exceptions	You are at a quota. The first thing to alarm on
Invocation latency	Time to respond. Watch percentiles, not the mean
Model invocation logs	The full request and response. Off by default, from module 16

ON THE EXAM

Alarm on throttling exceptions. It is the one signal that means customers are being refused rather than merely waiting.

On-demand or Provisioned Throughput?

ON-DEMAND, WHEN

Traffic is uncertain

- Volume varies or is unknown.
- No commitment is acceptable.
- Throttling can be handled by retry.
- You are still learning the workload.

PROVISIONED, WHEN

Throughput must be guaranteed

- Sustained, predictable, high volume.
- Throttling is unacceptable.
- You can commit to a term for the discount.
- You have measured the tokens per minute you need.

THE TIEBREAKER

Measure first. Buying model units without a load test is buying an unknown quantity of an unfamiliar unit.

Custom model deployment

- Fine-tune with the Bedrock or SageMaker AI customization API, then **CreateCustomModel**.
- Then **CustomModelDeployment** deploys it for **on-demand inference**.
- This is the path that makes the old rule wrong. A custom model does **not** automatically need Provisioned Throughput.
- Support is model-specific, so check before designing around it. Module 11 covered the conditions.
- Each deployment carries its own RPM and TPM limits.

> ON THE EXAM
>
> Modules 11 and 16 both flagged this. Neither 'custom always needs Provisioned Throughput' nor the reverse is a safe rule.

Rate limits per custom model deployment

BASE MODEL	RPM AND TPM PER DEPLOYMENT
Amazon Nova Micro	2,000 RPM, 4,000,000 TPM
Amazon Nova Lite	2,000 RPM, 4,000,000 TPM
Amazon Nova Pro	200 RPM, 800,000 TPM
Latency note	Expect a 20 to 55 percent Time To First Token difference against the base model

ON THE EXAM

The last row is the one to remember: invoking a fine-tuned adapter is measurably slower to first token than invoking the base model.

Agentic workloads multiply token use

- One user request can become **many model calls**: plan, call a tool, read the result, decide again.
- So token consumption scales with **steps**, not with users. Capacity planning changes shape.
- Conversation history is resent each step, from module 16, so later steps cost more than earlier ones.
- A loop that does not terminate consumes quota until something stops it.
- Module 27 builds agents. Plan quota for the worst case, not the median.

ON THE EXAM

Estimating capacity from request count badly under-counts an agentic workload. Estimate from expected steps per request.

Custom Model Import

- Import model weights trained elsewhere and serve them through Bedrock.
- Capacity is measured in **Custom Model Units**, determined **at import time**.
- CMUs depend on **architecture, parameter count and context length**. AWS gives Llama 3.1 8B 128K as 2 CMUs, and 70B 128K as 8.
- **Default maximum of three model copies** per account per imported model. Raisable via Service Quotas.
- Availability is limited by Region. Check before planning around it.

> ON THE EXAM
>
> Module 16's limitation still applies: the Converse API does not support imported models, so portable code needs the Invoke API instead.

Custom Model Import billing

- Billed in **five-minute windows**, for as long as a model copy is active.
- Billing starts from the **first successful invocation**, not from import.
- So an imported model that nobody calls is not costing you model copy time.
- Copies scale with demand, up to your quota, and each active copy bills.
- Throughput and concurrency per copy are determined during the import workflow.

> ON THE EXAM
>
> The five-minute window and the first-invocation start make this behave more like serverless than like Provisioned Throughput.

Custom Model Import or a SageMaker AI endpoint?

IMPORT TO BEDROCK

You want the tooling

- Guardrails, Knowledge Bases, Evaluations.
- No instances to size or scale.
- Supported architectures only.
- No Converse API support.

SAGEMAKER AI ENDPOINT

You want control

- Any architecture, any container.
- Inferentia, Trainium, specific instances.
- Full inference optimisation toolkit.
- You own scaling and capacity.

THE TIEBREAKER

Import buys you the Bedrock ecosystem. An endpoint buys you module 22's levers. Both serve the same weights.

Foundation models on SageMaker AI

- **JumpStart** deploys open foundation models to a SageMaker AI endpoint in a few clicks.
- **Large Model Inference containers** handle the serving: sharding, batching, and generation.
- This is where module 22's optimisation toolkit applies, and Bedrock's managed path does not.
- You get instance choice, including Inferentia and Trainium, and full control of scaling.
- You also get all of modules 21 to 24 to operate. That is the cost of the control.

ON THE EXAM

The same open model can be served three ways: imported to Bedrock, deployed via JumpStart, or containerised yourself.

Bedrock or SageMaker AI, for a foundation model

YOU NEED	THE ANSWER
A managed model, fast, with no infrastructure	Bedrock
Guardrails, Knowledge Bases, Evaluations	Bedrock
A model Bedrock does not offer, supported architecture	Bedrock Custom Model Import
Inferentia, Trainium, or specific instance types	SageMaker AI endpoint
Compilation, quantization, speculative decoding	SageMaker AI endpoint
Full control of scaling, batching and the container	SageMaker AI endpoint

ON THE EXAM

Bedrock manages capacity and gives you an ecosystem. SageMaker AI gives you levers and asks you to operate them.

Which deployment path

WHAT THE SCENARIO SAYS	THE ANSWER
Variable or unknown traffic on a base model	On-demand
Sustained high volume, throttling unacceptable	Provisioned Throughput
Throttled at peak, traffic still variable	Backoff, cross-Region profile, quota increase
A model fine-tuned on Bedrock or SageMaker AI	Custom model deployment, on demand where supported
Open weights not offered on Bedrock	Custom Model Import, or a SageMaker AI endpoint
Need Inferentia, or the optimisation toolkit	SageMaker AI endpoint, not Bedrock
Need Guardrails and Knowledge Bases	Bedrock, so import rather than an endpoint

ON THE EXAM

Row three is the common one. Throttling is usually a quota conversation, not a reason to commit to reserved capacity.

Resilience across Regions

- **Cross-Region inference profiles** route within a geography, which helps with capacity and with Region issues.
- They are not a disaster recovery design. They are a throughput and availability aid.
- True multi-Region resilience means model access and quotas in both Regions, and routing in front.
- **Provisioned Throughput is Region-specific.** Capacity in one Region does not serve another.
- And residency rules may forbid the whole approach, which is module 08's tension again.

> ON THE EXAM
>
> Reserved capacity does not travel. A multi-Region design with Provisioned Throughput means buying it twice.

Bedrock capacity options move

- Bedrock's pricing and capacity structures have changed more than once, and will again.
- **Learn the shape of the decision**: pay per use, or reserve throughput, or bring your own hosting.
- That shape survives. The specific product names and hourly rates do not.
- Check current options in the console for your Region before committing budget.
- The same advice as module 12 gave about the model catalogue, applied to capacity.

ON THE EXAM

Anything quoting a specific hourly model unit price is worth checking. Those tables go stale faster than the concepts do.

Switching models safely

- Run your evaluation set against the new model first, from module 18.
- **Prompts are tuned to a model version**, so re-test them rather than assuming they transfer.
- Route a small share of traffic first, using your own application logic.
- Bedrock has no deployment guardrails. **Module 24's canary and rollback do not apply here.**
- Keep the previous model ID configured, so reverting is one change rather than a rebuild.

> ON THE EXAM
>
> This is a real gap between the two worlds. On an endpoint, SageMaker AI manages the cutover; on Bedrock, your application does.

How each path bills

PATH	COST SHAPE
On-demand	Per input and output token. No floor, and no ceiling
Provisioned Throughput	Per model unit-hour, whether used or not. Term discounts
Custom model deployment	Per token, within the deployment's RPM and TPM limits
Custom Model Import	Per five-minute window per active copy, from first invocation
SageMaker AI endpoint	Per instance-hour, continuously. Module 20
Batch inference	Cheaper per token, and separate quotas

ON THE EXAM

Only two of these have a floor you pay when idle: Provisioned Throughput and a SageMaker AI endpoint. That is the real comparison.

Attributing foundation model cost

- **Tag Provisioned Throughput** when you create it, so the reserved capacity lands on a cost centre.
- On-demand spend is harder to attribute, because many callers share one account quota.
- **requestMetadata** from module 16 lets you tag invocations for later analysis.
- Application-side accounting is usually needed: count tokens per tenant or per feature.
- Without it, a runaway prompt in one feature is invisible inside a single account total.

> ON THE EXAM
>
> An account-level token bill tells you the total and nothing about the cause. Attribution has to be designed in.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Throttling exceptions at peak	TPM or RPM quota reached. Backoff, spread, or request more
Switching API did not help throttling	Quotas are the combined sum across all four invoke APIs
Quota increase request denied	Existing allocation not being consumed
Cannot buy committed Provisioned Throughput	Account has no model units. Request them through Support
Provisioned Throughput cannot be deleted	A committed term is still running. Billing continues
Fine-tuned model slower to first token	Expected: 20 to 55 percent TTFT difference on the adapter
Converse API fails on an imported model	Not supported. Use the Invoke API

ON THE EXAM

Row six is documented behaviour, not a fault. Teams report it as a regression and it is simply how adapters work.

Measure the throughput you need before you buy any of it.

A customer service assistant on Bedrock receives throttling exceptions during a two-hour window each weekday afternoon. Outside that window, usage is well under quota.

Overall volume is growing month on month but the team cannot yet predict where it will settle. Finance will not approve a multi-month commitment.

What should they do?

A. Purchase Provisioned Throughput with a one-month commitment

B. Move the workload to batch inference to avoid the quota

C. Add exponential backoff, use a cross-Region inference profile, and request a quota increase

D. Switch from the Converse API to InvokeModel to use a separate quota

◆ PAUSE AND DECIDE

What should they do?

A Purchase Provisioned Throughput with a one-month commitment

Finance has ruled out a commitment, and the team cannot yet size the capacity. Committed PT cannot be deleted before the term ends.

B Move the workload to batch inference to avoid the quota

Batch does have separate quotas, and this is a customer service assistant. Somebody is waiting, so batch is the wrong shape entirely.

C Add exponential backoff, use a cross-Region inference profile, and request a quota increase

All three address a peak-hour quota problem without committing to capacity, and consuming the existing allocation strengthens the increase request.

D Switch from the Converse API to InvokeModel to use a separate quota

The quotas are the combined sum across Converse, ConverseStream, InvokeModel and InvokeModelWithResponseStream. Switching API changes nothing.

THE SIGNAL

Throttling in a two-hour window with unpredictable growth is a quota conversation, not a capacity purchase.

A document processing platform runs a steady 4 million tokens per minute through one Bedrock model, twenty-four hours a day, and has done for eight months. Volume varies by less than 10 percent.

Throttling has caused two customer-visible incidents. The business wants guaranteed throughput and will commit to a term for a lower rate.

What should they move to?

A Provisioned Throughput, sized from measured consumption, with a term commitment

B A cross-Region inference profile to spread the load

C Custom Model Import, to get dedicated model copies

D A SageMaker AI endpoint with auto scaling

◆ PAUSE AND DECIDE

What should they move to?

A Provisioned Throughput, sized from measured consumption, with a term commitment

Sustained, predictable, high volume with throttling unacceptable and a willingness to commit. Eight months of data makes the sizing defensible.

B A cross-Region inference profile to spread the load

Raises effective throughput and still shares a quota. It does not guarantee capacity, which is what the business asked for.

C Custom Model Import, to get dedicated model copies

For weights you bring. They are using a Bedrock base model, so there is nothing to import.

D A SageMaker AI endpoint with auto scaling

Means self-hosting a model Bedrock already serves, taking on instances, scaling and optimisation to solve a quota problem.

THE SIGNAL

Contrast with scenario 1. Same symptom, opposite answer, because the traffic is predictable and the business will commit.

> A research team has trained an open-weight model with a supported architecture. They want to serve it alongside their other Bedrock work, using Bedrock Guardrails and a Knowledge Base, with no instances to manage.
>
> Their existing application code is written against the Converse API.

What should they do, and what must they change?

A. Deploy to a SageMaker AI endpoint, since Bedrock cannot host external weights

B. Use Custom Model Import, and keep the Converse API as-is

C. Use Provisioned Throughput on a similar Bedrock base model instead

D. Use Custom Model Import, and switch that model's calls to the Invoke API

◆ PAUSE AND DECIDE

What should they do, and what must they change?

A Deploy to a SageMaker AI endpoint, since Bedrock cannot host external weights

The premise is wrong: Custom Model Import exists for exactly this. An endpoint also means managing instances, which they explicitly do not want.

B Use Custom Model Import, and keep the Converse API as-is

Right service, wrong assumption. The Converse API does not support imported models, so those calls would fail.

C Use Provisioned Throughput on a similar Bedrock base model instead

Abandons the model they trained and serves a different one. Provisioned Throughput is a capacity option, not a hosting route for external weights.

D Use Custom Model Import, and switch that model's calls to the Invoke API

Import gives them Bedrock's tooling with no instances, and switching to Invoke resolves the documented Converse limitation.

THE SIGNAL

B and D differ only in whether you know the Converse limitation. That single fact from module 16 separates them.

A team has fine-tuned Amazon Nova Lite for a classification task. An engineer states that the fine-tuned model will require Provisioned Throughput to serve, and asks finance to approve a six-month commitment.

Expected traffic is around 300 requests per minute, well within published on-demand limits for a custom deployment.

How should you respond?

A. Agree. All custom models require Provisioned Throughput

B. Correct it. Deploy on demand with CustomModelDeployment, and measure before committing

C. Agree, but recommend no commitment rather than six months

D. Recommend Custom Model Import instead, to avoid the commitment

◆ PAUSE AND DECIDE

How should you respond?

A Agree. All custom models require Provisioned Throughput

A rule that used to hold and no longer does. On-demand custom model deployment exists for supported Nova models.

B Correct it. Deploy on demand with CustomModelDeployment, and measure before committing

Nova Lite custom deployments allow 2,000 RPM. At 300 RPM they are well inside it, and on-demand gives the measurement a commitment would need.

C Agree, but recommend no commitment rather than six months

Better than six months, and still reserves capacity they do not need and cannot yet size.

D Recommend Custom Model Import instead, to avoid the commitment

Import is for weights trained outside Bedrock. A Nova model fine-tuned through Bedrock is already there; importing it is not the path.

THE SIGNAL

A six-month commitment cannot be cancelled. Getting this wrong is not a configuration error, it is a binding financial one.

What to take from this module

- **A model unit is tokens per minute**, input and output combined. Not a machine.
- **Committed Provisioned Throughput cannot be deleted early**, and billing runs until you do.
- **TPM and RPM are the combined sum** across all four invoke APIs. Switching API changes nothing.
- **Custom models can be served on demand** where supported. The old blanket rule is wrong.
- **Measure on demand first**, then reserve. You cannot size capacity you have never used.

> ON THE EXAM
>
> Module 26 moves to retrieval in production: Bedrock Knowledge Bases, retrieval configuration and reranking.